

Authly

Plataforma de Autenticación y Autorización

Contrato API Externa

Versión 2.0

Fecha: Octubre 2026
Equipo Backend: Valentino Bustamante, Felipe Monetti

Cambios respecto a v1.0: roles en JWT, recuperación de contraseña de usuario (CU-11), verificación de permisos (CU-09) actualizada, paginación en listados, gestión de roles (CU-04), gestión de usuarios (CU-06), recuperación de contraseña de admin (CU-10), formato de error unificado en panel.

1. Introducción

Este contrato describe el comportamiento de la API de Authly en su versión 2.0. Incluye tanto los endpoints de la API externa (consumidos por aplicaciones de terceros bajo /api/auth/) como los endpoints del panel de administración (/api/platform/ y /api/recovery/).

NOTA: Algunos endpoints pueden no estar disponibles todavía. Se comunicará por WhatsApp cuando cada endpoint esté listo para consumir.

Authly provee a las aplicaciones consumidoras un sistema unificado para:

- Registro y verificación de usuarios
- Inicio de sesión con generación de token JWT
- Verificación de permisos por aplicación
- Recuperación de contraseña (usuarios y administradores)
- Gestión de roles, permisos y usuarios desde el panel

Base URL: Se comunicará por WhatsApp cuando el deploy esté listo.

2. Mecanismos de Autenticación

2.1. Credenciales de Aplicación (client_id / client_secret)

Cada aplicación registrada en Authly recibe un par de credenciales:

- client_id: identificador público (UUID). Se puede almacenar en el frontend.
- client_secret: clave secreta. Se muestra UNA SOLA VEZ al momento de crear o regenerar. Debe almacenarse de forma segura en el backend de la aplicación consumidora. Authly almacena solo el hash (SHA-256).

Uso: Se envían como headers X-Client-Id y X-Client-Secret en las llamadas que requieren identificar a la aplicación.

2.2. JWT con RS256 (firma asimétrica)

Cuando un usuario inicia sesión, Authly genera un JSON Web Token (JWT) firmado con RS256.

Contenido del token (payload)

Campo	Tipo	Descripción
sub	string (UUID)	ID del usuario
type	string	Tipo de token: "admin" o "user"
org_id	string (UUID)	ID de la organización
app_id	string (UUID)	ID de la aplicación (solo en tokens de usuario)
roles	string[]	Lista de nombres de rol del usuario (solo en tokens de usuario). Hint para UI, no usar para autorización.
iat	number	Timestamp de emisión (Unix)
exp	number	Timestamp de expiración (8 horas después de iat)

IMPORTANTE: El campo roles es un hint para que la UI del sistema externo pueda mostrar pantallas según el rol del usuario. La autorización real debe hacerse siempre consultando el endpoint /api/auth/permissions/verify (CU-09).

Características del token

- Duración: 8 horas (28800 segundos).
- No hay refresh token en esta versión.
- No hay mecanismo de revocación en esta versión.
- El token NO contiene permisos individuales. Los permisos se verifican con el endpoint /api/auth/permissions/verify.

2.3. Clave Pública

La clave pública RSA para verificar los tokens está disponible en:

- Endpoint GET /api/auth/public-key
- Archivo public_key.pem compartido por WhatsApp

3. API Externa (/api/auth/)

Endpoints consumidos por las aplicaciones externas. Todos los errores responden con el formato:

```
{ "detail": "Mensaje descriptivo." }
```

3.1. Registrar Usuario — Paso 1 (CU-05)

Método	URL
POST	/api/auth/register

Headers requeridos

Header	Tipo	Descripción
X-Client-Id	string (UUID)	Client ID de la app
X-Client-Secret	string	Client Secret de la app

Request Body

```
{
  "nombre": "Juan",
  "apellido": "Pérez",
  "email": "juan@ejemplo.com",
  "password": "MiClave123!@"
}
```

Validaciones

- Email: formato válido, no registrado previamente en la misma aplicación.
- Password: mínimo 10 caracteres, al menos 1 mayúscula, 1 minúscula, 1 número, 1 carácter especial.
- Nombre y apellido: obligatorios, no vacíos.

Respuestas

Código	Estado	Descripción
201	Created	Se envió código de verificación al email.
400	Bad Request	Datos inválidos (email, password débil, nombre/apellido vacíos).
401	Unauthorized	client_id o client_secret inválidos.
409	Conflict	El email ya está registrado en esta aplicación.
429	Too Many Requests	Límite de códigos excedido.

Respuesta exitosa (201)

```
{
  "message": "Se envió código de verificación al email.",
  "expires_in": 600
}
```

3.2. Verificar Código de Registro — Paso 2 (CU-05)

Método	URL
POST	/api/auth/register/verify

Headers requeridos

Header	Tipo	Descripción
X-Client-Id	string (UUID)	Client ID de la app
X-Client-Secret	string	Client Secret de la app

Request Body

```
{
  "nombre": "Juan",
  "apellido": "Pérez",
  "email": "juan@ejemplo.com",
  "password": "MiClave123!@",
  "codigo": "482910"
}
```

Restricciones del código

- 6 dígitos numéricos.
- Expira a los 10 minutos de generado.
- Máximo 5 intentos de validación por código.
- Máximo 3 códigos por hora por combinación email + propósito + aplicación.

Respuestas

Código	Estado	Descripción
200	OK	Cuenta verificada y activada. Se asigna rol por defecto.
400	Bad Request	Código incorrecto, expirado o intentos agotados.
401	Unauthorized	client_id o client_secret inválidos.
404	Not Found	No existe un código pendiente para ese email.
409	Conflict	El email ya está registrado en esta aplicación.
422	Unprocessable	La aplicación no tiene rol por defecto configurado.

Respuesta exitosa (200)

```
{ "message": "Cuenta verificada exitosamente." }
```

3.3. Iniciar Sesión (CU-07)

Método	URL
POST	/api/auth/login

Headers requeridos

Header	Tipo	Descripción
X-Client-Id	string (UUID)	Client ID de la app
X-Client-Secret	string	Client Secret de la app

Request Body

```
{
  "email": "juan@ejemplo.com",
  "password": "MiClave123!@"
}
```

Respuestas

Código	Estado	Descripción
200	OK	Login exitoso. Retorna el JWT con roles incluidos.
401	Unauthorized	Credenciales de usuario o de aplicación inválidas.

Respuesta exitosa (200)

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 28800
}
```

El payload del JWT incluye el campo roles con la lista de nombres de rol del usuario. Ejemplo de payload decodificado:

```
{
  "sub": "a1b2c3d4-...",
  "type": "user",
  "org_id": "e5f6g7h8-...",
  "app_id": "i9j0k1l2-...",
  "roles": ["editor", "moderador"],
  "iat": 1730000000,
  "exp": 1730028800
}
```

Uso del token

El token debe enviarse en el header Authorization de cada request protegido:

`Authorization: Bearer eyJhbGciOiJSUzI1NiIs...`

3.4. Obtener Clave Pública (CU-08)

Método	URL
GET	/api/auth/public-key

Endpoint público, no requiere autenticación.

Respuesta (200)

```
{
  "public_key": "-----BEGIN PUBLIC KEY-----\nMIIBIjAN..."
}
```

3.5. Verificar Permiso de Usuario (CU-09)

Método	URL
POST	/api/auth/permissions/verify

Headers requeridos

Header	Tipo	Descripción
X-Client-Id	string (UUID)	Client ID de la app
X-Client-Secret	string	Client Secret de la app

Request Body

```
{
  "user_id": "a1b2c3d4-...",
  "resource": "documentos",
  "action": "leer"
}
```

resource y action se comparan en minúsculas y sin espacios en los extremos, igual que como se almacenan en la base.

Respuestas

Código	Estado	Descripción
200	OK	Retorna si el usuario tiene o no el permiso.
400	Bad Request	Falta user_id, resource o action.
401	Unauthorized	Credenciales de aplicación inválidas.
404	Not Found	El usuario no existe en esta aplicación.

Respuesta exitosa (200)

```
{ "allowed": true }
```

Si el usuario no tiene el permiso, retorna:

```
{ "allowed": false }
```

3.6. Recuperar Contraseña de Usuario — Paso 1 (CU-11)

Método	URL
POST	/api/auth/password/recover

Headers requeridos

Header	Tipo	Descripción
X-Client-Id	string (UUID)	Client ID de la app
X-Client-Secret	string	Client Secret de la app

Request Body

```
{
  "email": "juan@ejemplo.com"
}
```

La respuesta es siempre la misma, exista o no el email. Esto evita revelar si una cuenta está registrada.

Respuestas

Código	Estado	Descripción
200	OK	Respuesta genérica (no revela si el email existe).
401	Unauthorized	Credenciales de aplicación inválidas.
429	Too Many Requests	Límite de códigos excedido.
500	Server Error	No se pudo enviar el código.

Respuesta (200)

```
{ "message": "Si el email está registrado, se envió un código de recuperación." }
```

3.7. Resetear Contraseña de Usuario — Paso 2 (CU-11)

Método	URL
POST	/api/auth/password/reset

Headers requeridos

Header	Tipo	Descripción
X-Client-Id	string (UUID)	Client ID de la app
X-Client-Secret	string	Client Secret de la app

Request Body

```
{
  "email": "juan@ejemplo.com",
```

```
  "codigo": "482910",
  "new_password": "NuevaClave456!@"
}
```

Reglas

- El código se valida primero. Si es incorrecto, consume un intento.
- La contraseña se valida después. Si no cumple complejidad, NO consume intento del código.
- La contraseña nueva debe ser distinta de la actual.

Respuestas

Código	Estado	Descripción
200	OK	Contraseña actualizada exitosamente.
400	Bad Request	Código incorrecto/expirado/intentos agotados, o contraseña inválida/igual a la actual.
401	Unauthorized	Credenciales de aplicación inválidas.
404	Not Found	No hay código pendiente, o usuario no encontrado.

Respuesta exitosa (200)

```
{ "message": "Contraseña actualizada exitosamente." }
```

4. API del Panel de Administración (/api/platform/)

Endpoints consumidos por el panel web de Authly. Todos requieren JWT de administrador en el header Authorization, salvo /api/recovery/ que es público.

4.1. Formato de Error Unificado

Todas las rutas del panel responden con este formato:

```
{
  "detail": "Mensaje en español.",
  "campo": "general",
  "reinicio": false
}
```

- campo: indica qué input marcar en el formulario (nombre, apellido, roles, permisos, codigo, pass). Por defecto "general".
- reinicio: si es true, el frontend debe reiniciar el flujo.
- Algunos errores incluyen claves extra: motivo, cantidad_usuarios, rol_id.

Valores de motivo

motivo	Cuándo
aplicacion-inexistente	La aplicación no existe o es de otra organización
rol-inexistente	El rol no existe (en usuarios, trae además rol_id)
usuario-inexistente	El usuario no existe
tiene-usuarios	Se intenta borrar una app o rol con usuarios (trae cantidad_usuarios)

Excepciones: Los endpoints /auth/ (API externa) conservan el formato simple {"detail": "..."}. Los errores 422 de validación de formato usan el formato por defecto de FastAPI.

4.2. Gestionar Aplicaciones (CU-03)

Base: /api/platform/applications/

Listar aplicaciones

Método	URL
GET	/api/platform/applications/?pagina=1&tamanio=10

Parámetros de query: pagina (default 1), tamanio (default 10). Ordenado por nombre ascendente.

Respuesta (200)

```
{
  "aplicaciones": [
    {
      "client_id": "uuid",
      "nombre": "Mi App",
      "descripcion": "...",
      "creado_en": "2026-10-01T12:00:00",
```

```
      "cantidad_usuarios": 15
    }
  ],
  "pagina": 1,
  "tamanio": 10,
  "total_apps": 3,
  "total_usuarios_organizacion": 42
}
```

Obtener aplicación individual

Método	URL
GET	/api/platform/applications/{client_id}

Respuesta (200)

```
{
  "client_id": "uuid",
  "nombre": "Mi App",
  "descripcion": "...",
  "creado_en": "2026-10-01T12:00:00",
  "cantidad_usuarios": 15
}
```

Crear aplicación

Método	URL
POST	/api/platform/applications/

Body: { "nombre" (obligatorio, máx 100), "descripcion" (opcional, máx 500) }

Respuesta 201: { "nombre", "client_id", "client_secret" }. El client_secret se muestra una sola vez.

Editar aplicación

Método	URL
PUT	/api/platform/applications/{client_id}

Body: { "nombre", "descripcion" }. Nombre único dentro de la organización.

Regenerar secret

Método	URL
POST	/api/platform/applications/{client_id}/regenerar-secret

Respuesta 200: { "client_id", "client_secret" }. El nuevo secret se muestra una sola vez.

Eliminar aplicación

Método	URL
DELETE	/api/platform/applications/{client_id}

Respuesta 204 si se eliminó. Si la aplicación tiene usuarios asociados, responde 409:

```
{
  "detail": "No se puede eliminar: la aplicación tiene 5 usuario(s) asociado(s).",
  "campo": "general",
  "reinicio": false,
  "motivo": "tiene-usuarios",
  "cantidad_usuarios": 5
}
```

Errores comunes de aplicaciones

Código	Situación
400	campo: nombre — nombre vacío
404	motivo: aplicacion-inexistente
409	campo: nombre — nombre duplicado en la org
409	motivo: tiene-usuarios — DELETE con usuarios
422	Longitud excedida o tipo incorrecto

4.3. Gestionar Roles y Permisos (CU-04)

Base: /api/platform/applications/{client_id}/roles/

Todos validan que la aplicación pertenezca a la organización del admin autenticado.

Método	Ruta	Respuesta
GET	/	200 — { "roles": [Rol] }
POST	/	201 — { "rol": Rol, "rol_por_defecto_anterior": nombre | null }
PUT	/{rol_id}	200 — igual que POST
DELETE	/{rol_id}	200 — { "id", "nombre", "por_defecto" }

Objeto Rol

```
{
  "id": "uuid",
  "nombre": "Editor",
  "descripcion": "Puede editar contenido",
  "por_defecto": false,
  "cantidad_usuarios": 3,
  "permisos": [
    { "recurso": "documentos", "accion": "editar" },
    { "recurso": "documentos", "accion": "leer" }
  ]
}
```

Body para crear/editar rol

```
{
  "nombre": "Editor",
  "descripcion": "Puede editar contenido",
  "por_defecto": false,
  "permisos": [
    { "recurso": "documentos", "accion": "editar" }
  ]
}
```

Reglas

- Nombre: obligatorio, máximo 100 caracteres, único dentro de la app (case insensitive).
- Descripción: opcional, máximo 500 caracteres.
- Permisos: recurso y acción obligatorios (máx 100 c/u), se guardan en minúsculas, sin duplicados.
- Al marcar un rol como por_defecto, el anterior se desmarca automáticamente. Su nombre vuelve en rol_por_defecto_anterior.
- No se puede eliminar un rol que tiene usuarios asignados (409 con cantidad_usuarios).

Errores

Código	Situación

400	campo: nombre (vacío) o campo: permisos (vacío, repetido, muy largo)
404	motivo: rol-inexistente o aplicacion-inexistente
409	campo: nombre (nombre duplicado)
409	motivo: tiene-usuarios + cantidad_usuarios (DELETE)

4.4. Gestionar Usuarios de una Aplicación (CU-06)

Base: /api/platform/applications/{client_id}/users/

Método	Ruta	Respuesta
GET	/?pagina=1&tamanio=10	200 — { "usuarios": [Usuario], "total", "pagina", "tamanio" }
PUT	/{usuario_id}	200 — Usuario actualizado
DELETE	/{usuario_id}	200 — Usuario eliminado

Objeto Usuario

```
{
  "id": "uuid",
  "nombre": "Juan",
  "apellido": "Pérez",
  "email": "juan@ejemplo.com",
  "roles": [
    { "id": "uuid", "nombre": "Editor" }
  ]
}
```

Body para editar usuario (PUT)

```
{
  "nombre": "Juan",
  "apellido": "Pérez",
  "roles": ["uuid-rol-1", "uuid-rol-2"]
}
```

Reglas

- Listado ordenado por apellido y luego nombre, con criterio español.
- pagina ≥ 1, tamanio de 1 a 100 (fuera de rango: 422).
- PUT cambia nombre, apellido y roles en una transacción atómica.
- Nombre y apellido obligatorios (máximo 100). El email no se puede modificar.
- Roles: al menos uno, todos deben pertenecer a la aplicación. Repetidos se ignoran.
- DELETE elimina al usuario y sus asignaciones de rol.

Errores

Código	Situación

400	campo: nombre, apellido o roles (vacío o sin roles)
404	motivo: usuario-inexistente, rol-inexistente (con rol_id), o aplicacion-inexistente

4.5. Recuperar Contraseña de Administrador (CU-10)

Estos endpoints son públicos (no requieren JWT).

Solicitar código

Método	URL
POST	/api/recovery/codigo

Body: { "email" } (máximo 254 caracteres).

Respuesta siempre 200, no revela si el email existe:

```
{ "detail": "Si el correo corresponde a una cuenta, le enviamos un código." }
```

Confirmar y resetear contraseña

Método	URL
POST	/api/recovery/confirmar

```
{
  "email": "admin@ejemplo.com",
  "codigo": "482910",
  "password": "NuevaClave456!@"
}
```

Respuesta exitosa (200)

```
{ "detail": "Su contraseña fue actualizada." }
```

Reglas

- Códigos: 6 dígitos, vencen en 10 minutos, máximo 3 por hora, 5 intentos por código.
- Los errores de contraseña no consumen intentos ni gastan el código.
- La contraseña nueva debe ser distinta de la actual.
- Las sesiones abiertas antes del cambio siguen vigentes hasta vencer.

Errores de /recovery/confirmar (400)

- campo: codigo — mensaje único "Código inválido o vencido." (no distingue la causa).
- campo: pass — contraseña débil o igual a la actual.

4.6. Obtener Datos del Administrador Actual

Método	URL
GET	/api/me

Header: Authorization: Bearer <token> (JWT de admin).

Respuesta (200)

```
{
  "id": "uuid",
  "nombre": "Felipe",
  "apellido": "Monetti",
  "email": "felipe@ejemplo.com",
  "organizacion_id": "uuid",
  "nombre_organizacion": "Mi Organización"
}
```

5. Flujos de Integración

5.1. Registro → Verificación → Login

- 1. POST /api/auth/register con datos del usuario y credenciales de app.
- 2. Authly envía código de 6 dígitos al email.
- 3. El usuario ingresa el código en la aplicación.
- 4. POST /api/auth/register/verify con todos los datos y el código.
- 5. Authly crea al usuario con rol por defecto y activa la cuenta.
- 6. POST /api/auth/login con email y password.
- 7. Authly retorna JWT con roles incluidos. La aplicación lo almacena.

5.2. Verificar permisos antes de una acción

- 1. El backend recibe un request con JWT en Authorization.
- 2. Verificar el token localmente con la clave pública (recomendado).
- 3. Para verificar un permiso específico: POST /api/auth/permissions/verify con credenciales de app, user_id, resource y action.
- 4. Authly responde { "allowed": true/false }.
- 5. La aplicación permite o deniega la acción.

Nota sobre roles en el JWT: El campo roles del token es un hint para la UI (mostrar/ocultar pantallas). Para decisiones de autorización, siempre usar el endpoint /api/auth/permissions/verify, que consulta los permisos en tiempo real.

5.3. Recuperar contraseña de usuario

- 1. POST /api/auth/password/recover con email y credenciales de app.
- 2. Si el email existe, Authly envía código de 6 dígitos.
- 3. El usuario ingresa el código y la nueva contraseña.
- 4. POST /api/auth/password/reset con email, código, nueva contraseña y credenciales de app.
- 5. Authly actualiza la contraseña.

5.4. Verificación local del token

- 1. Obtener la clave pública (GET /api/auth/public-key o archivo .pem).
- 2. En cada request protegido, tomar el token del header Authorization.
- 3. Verificar la firma con RS256.
- 4. Si la firma es válida y no expiró, el usuario está autenticado.
- 5. Del payload: sub (user_id), type, org_id, app_id, roles.

Ejemplo en Python:

```
import jwt, requests
```

```
# Obtener clave pública (cachear)
resp = requests.get("http://<authly-url>/api/auth/public-key")
public_key = resp.json()["public_key"]

# Verificar token
payload = jwt.decode(token, public_key, algorithms=["RS256"])
user_id = payload["sub"]
roles = payload["roles"]  # hint para UI
```

6. Códigos de Error Comunes

Código	Significado	Qué hacer
400	Bad Request	Revisar el body. Algún campo está mal formado o falta.
401	Unauthorized	Verificar client_id/client_secret o que el JWT no esté expirado.
403	Forbidden	El usuario no tiene acceso (cuenta no verificada, sin permiso).
404	Not Found	El recurso no existe (usuario, código, aplicación, rol).
409	Conflict	Duplicado o restricción (email registrado, app con usuarios, nombre repetido).
422	Unprocessable	Los datos tienen formato correcto pero no cumplen reglas de negocio.
429	Too Many Requests	Límite de códigos excedido. Esperar e intentar más tarde.
500	Server Error	Error interno de Authly. Reportar al equipo.

7. Notas Importantes

- Todas las respuestas son en formato JSON.
- Los timestamps están en formato ISO 8601.
- Los IDs son UUID v4.
- Las rutas del panel usan formato de error unificado (sección 4.1). Las rutas de /auth/ usan formato simple.
- Ante cualquier duda, comunicarse por WhatsApp.

— Fin del documento —

Documento confidencial — Authly 2026